



机器学习导论

第2章 线性模型

2.1 线性回归

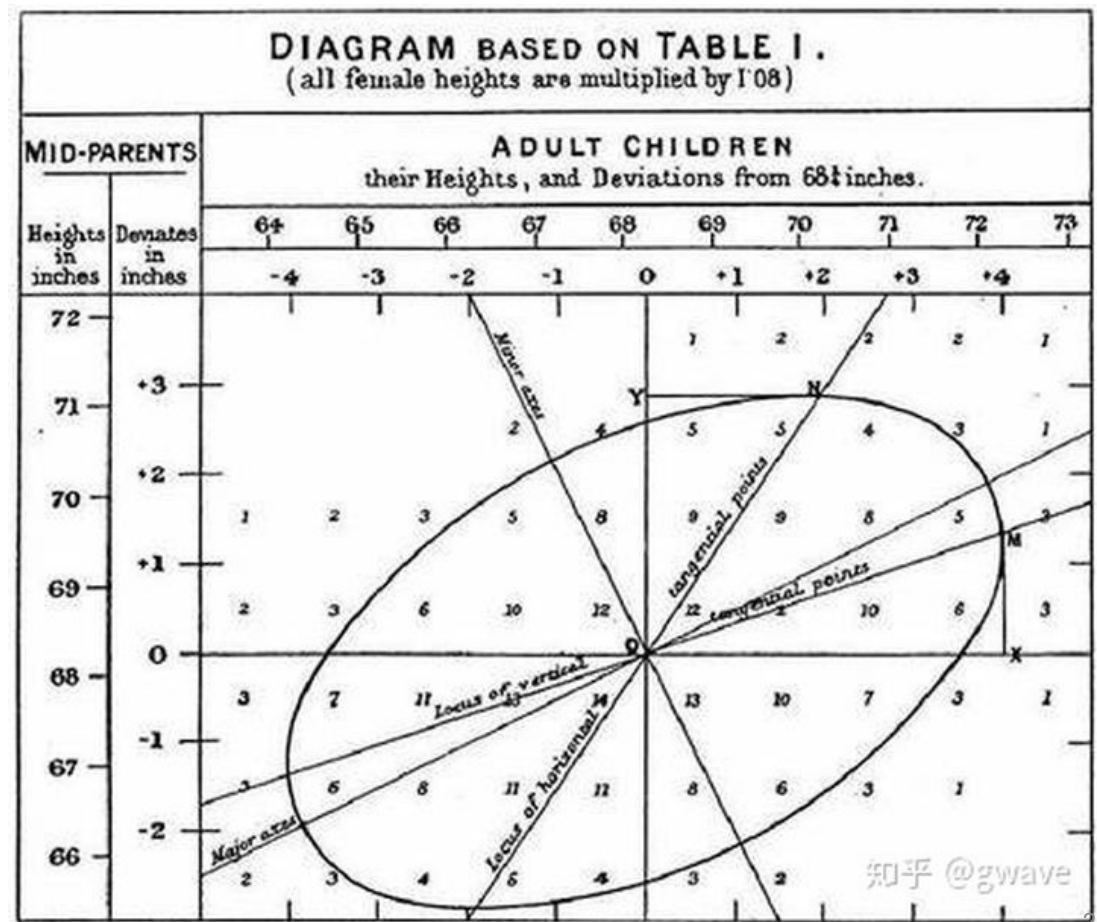
谢茂强

南开大学软件学院

本节内容主要来自于吴恩达Coursera网课

线性回归的历史

- 1855年，高尔顿(Francis Galton)发表《遗传的身高向平均数方向的回归》
- 他和学生Karl Pearson通过观察1078对夫妇，以每对夫妇的平均身高作为自变量，取他们的一个成年儿子的身高作为因变量，分析它们之间的关系，发现父母的身高和子女身高大致成**线性**关系。
- “即使父母的身高都‘极端’高，其子女“向平均数方向的**回归**”



目录

01. 预测任务、数据和预测模型

02. 模型的表示

03. 线性回归的代价函数（损失函数）

04. 优化算法：使用梯度下降法最小化代价函数

05. 以线性回归为例了解机器学习的主要工作和过程

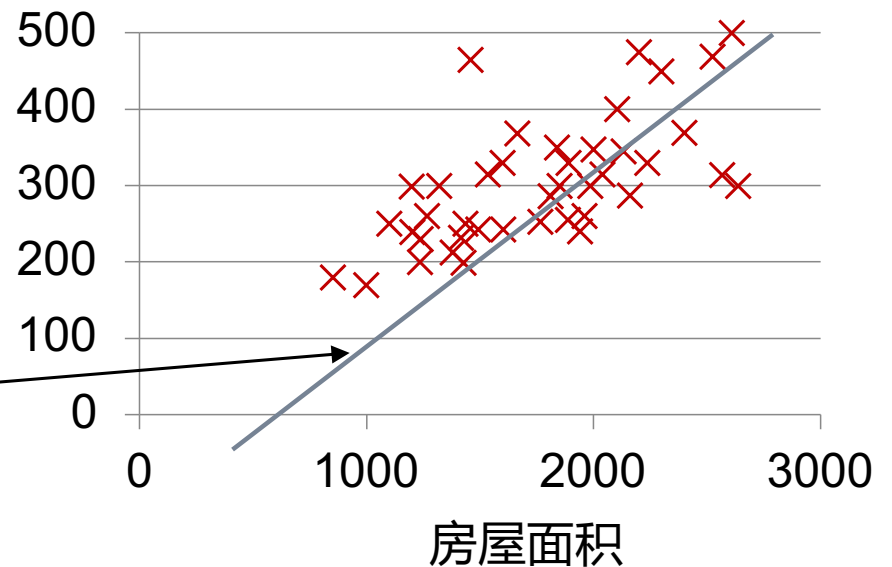
1. 线性回归

- 线性回归 (Linear regression) : 通过一组属性值的线性组合来预测目标值的方法
 - 用途: 定价 (房屋、债券、股票)、资信、物质成分浓度

线性回归的判决函数形如:

$$f_w(\mathbf{x}) = w_0 + w_1 x_1$$

房价



俄勒冈州波特兰市房价数据, 来自吴恩达Coursera课程

1. 任务: 基于房屋属性预测房价

- 房屋属性

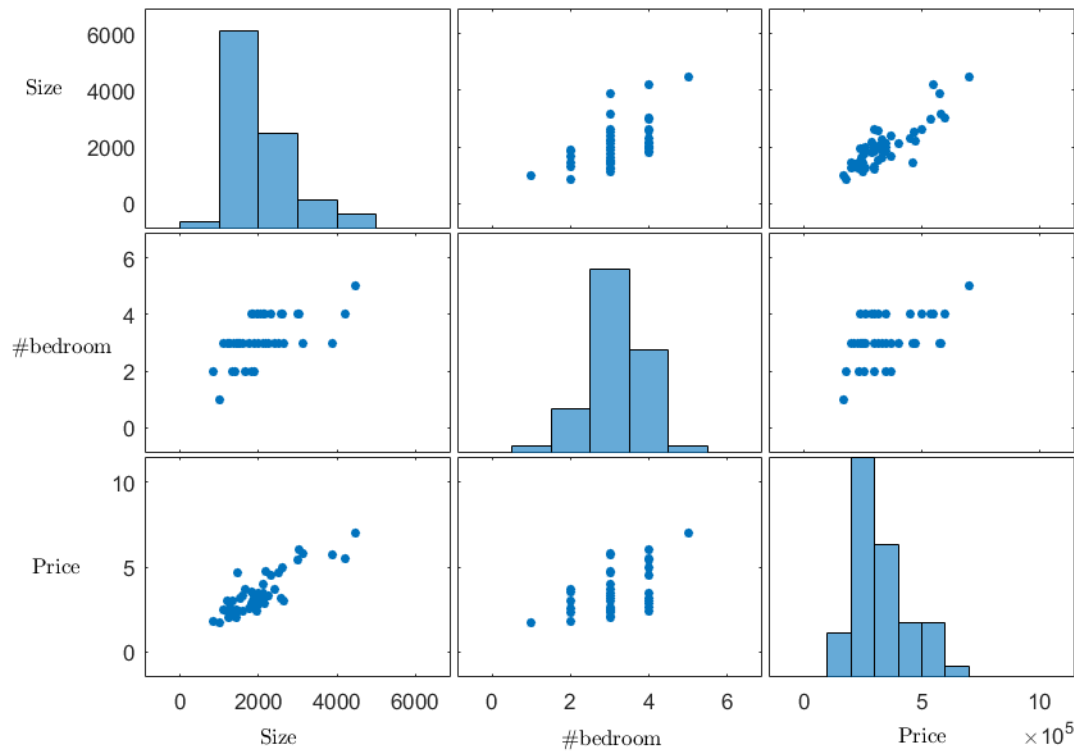
- Size,
- numbers of bedrooms,
- no. of floors,
- age of home

- 预测目标

- Price

房屋面积	房价(千元)
2104	460
1416	232
1534	315
852	178
...	...

房价数据集各维度间的相关系数



	Size	#Bedroom	Price
Size	1	0.559967	0.854988
#Bedroom	0.559967	1	0.442261
Price	0.854988	0.442261	1

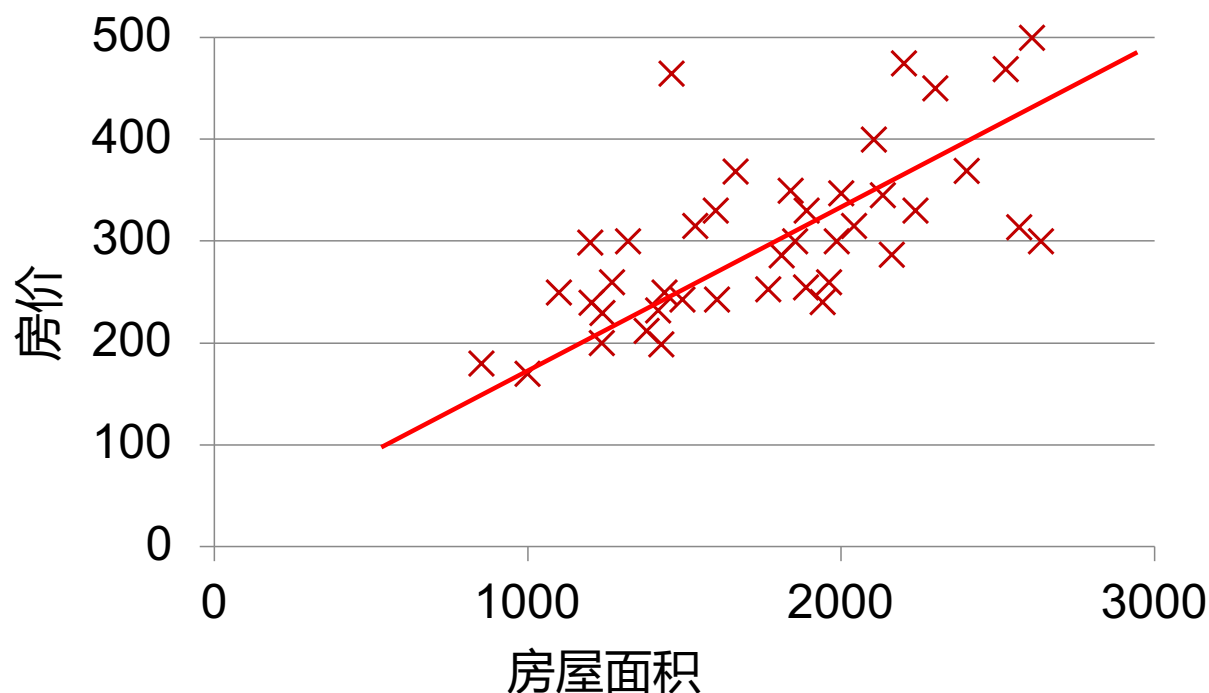
相关系数: Karl Pearson's correlation coefficient

1. 可视化房价与房屋面积间的关系

将 $\{(x^{(i)}, y^{(i)})\}$ 特征化后，每个样本看作特征空间中的一个点

总体上能够看到房价和房屋面积大致呈正比例关系

可否寻找一条直线尽可能去拟合样本分布，作为预测模型



目录

01. 预测任务、数据和预测模型

02. 模型的表示

03. 线性回归的代价函数（损失函数）

04. 优化算法：使用梯度下降法最小化代价函数

05. 以线性回归为例了解机器学习的主要工作和过程

2. 线性回归模型的表示

使用线性方程表示预测模型(或假设) h 。

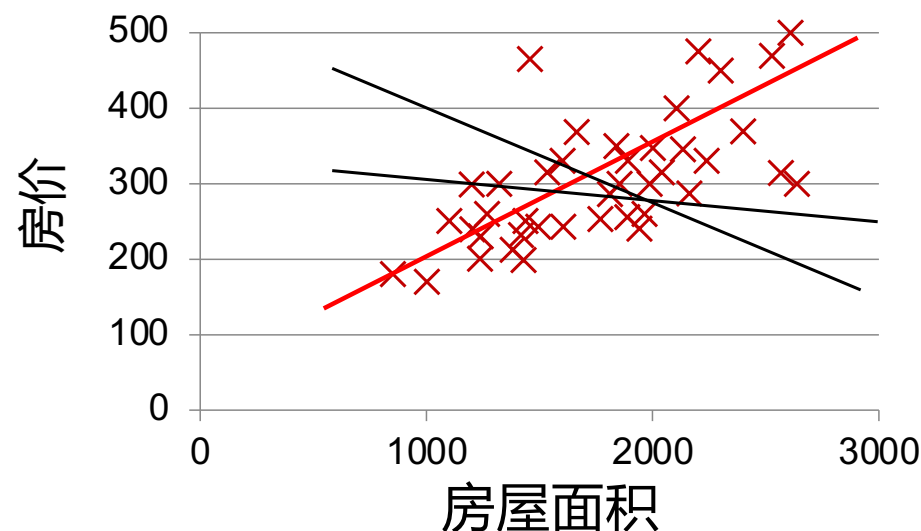
在这个模型里，不同的模型参数 θ_i 代表着不同的预测模型。

带来的问题：

1. 如何评估预测模型的好坏？
2. 如何使用训练数据评估预测模型的好坏？

模型 $f(x)$ (或假设 h) 使用线性方程表示：

$$f_w(\mathbf{x}) = w_0 + w_1 x_1$$

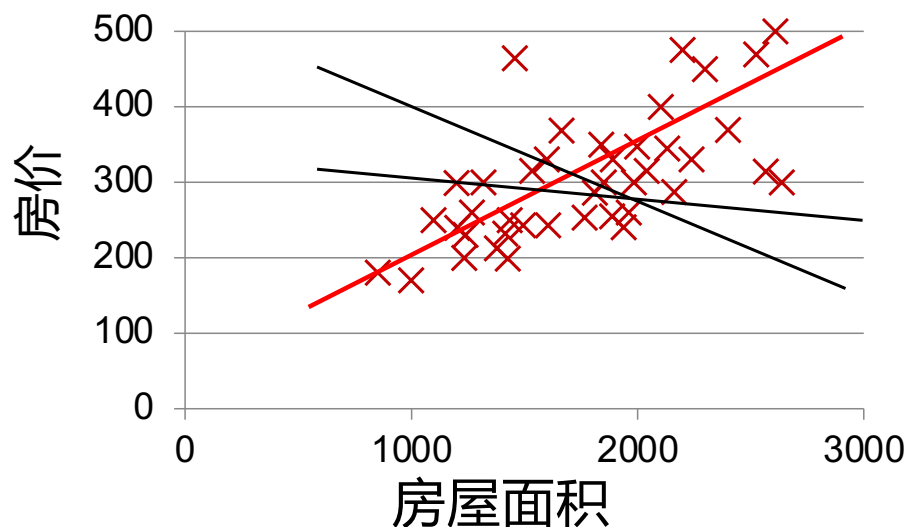


一元线性回归

2. 线性回归模型的表示

模型(或假设) h 使用线性方程表示

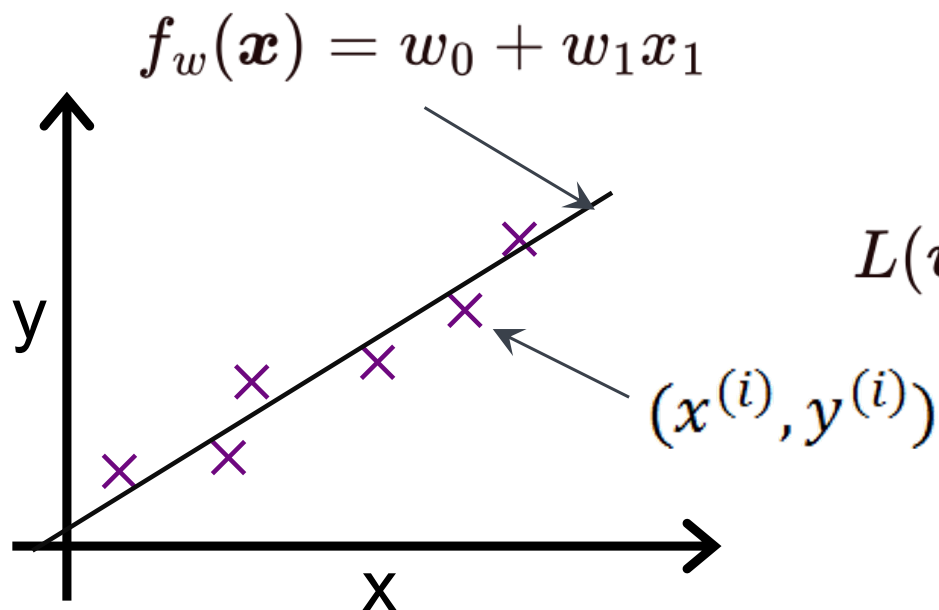
$$f_w(\mathbf{x}) = w_0 + w_1 x_1$$



通过历史训练数据获取合适的模型参数 θ

- 这个过程被称为机器学习的模型训练
- **输入**是训练数据
- **输出**是模型参数
- 典型训练策略是使用**代价函数**评估训练数据与模型参数的匹配程度，并为**参数调整提供支持**。

3. 使用代价函数去衡量拟合(或偏离)程度



代价函数(或损失函数):

$$L(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m \left(f_w(x^{(i)}) - y^{(i)} \right)^2$$

误差平方和

训练过程:

使 w_0, w_1 代表的线性模型 $f_w(\mathbf{x})$

$$\min_{w_0, w_1} L(w_0, w_1)$$

尽量拟合训练数据 $\{(x^{(i)}, y^{(i)})\}$

【注】：代价函数是关于模型参数的函数

符号表示法

Training set of housing prices (Portland, OR)

$m=41$

Size in feet² (x)

Price (\$) in 1000's (y)

2104

460

1416

232

1534

315

852

178

...

...

Notation:

m = Number of training examples

x 's = "input" variable / features

y 's = "output" variable / "target" variable

(X, y) : training sample

$(x^{(i)}, y^{(i)})$: i -th training sample

$x^{(1)} = 2104$

$x^{(2)} = 1416$

$y^{(1)} = 460$

$y^{(4)} =$

- 【常见表示方法】：小写字母：变量、小写字母加粗：向量、大写字母加粗：矩阵。但并不严格。
- 在周志华老师的《机器学习》和很多教材中，使用下标表示样本ID号。
- 在表示方法不严谨的时候，通过上下文判断具体表示。

4. 优化：最小化代价函数

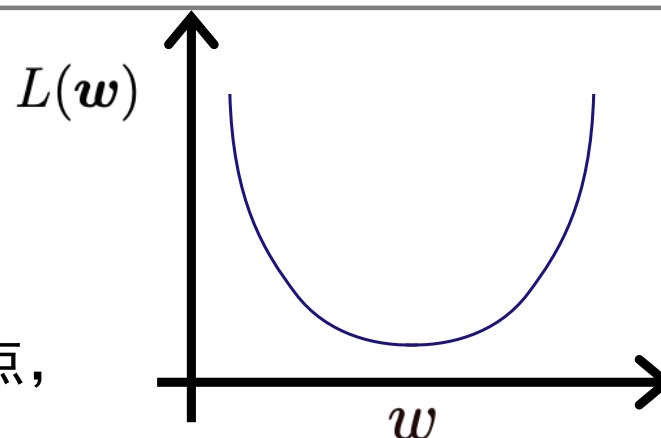
- 目标：使用带标注的训练数据在优化过程中选出最好的参数组合 (w_0, w_1)
- 方法：
 - 解析式求导
 - 梯度下降法

4.1 优化-求导法

损失函数是关于模型参数 w 的二次函数：

$$L(w) = aw^2 + bw + c$$

因为损失函数为二次多项式，因此，存在最低点，最低点出现在一阶导数为0处。



具体的损失函数为：

$$L(w) = \frac{1}{2m} \sum_{i=1}^m \left(f_w(x^{(i)}) - y^{(i)} \right)^2$$

然后 $L(w)$ 对每个 w_j 进行求导

即可求得损失函数最小时的模型参数 θ

$$w_j = (w_0, w_1, w_2, \dots, w_n)$$

$$\frac{\partial L(w)}{\partial w_j} = \dots = 0$$

4.1 优化-求导法-过程

$$\begin{aligned}\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}_j} &= \frac{\partial}{\partial \mathbf{w}_j} \frac{1}{2m} \sum_{i=1}^m (f_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)})^2 \\&= \frac{1}{2m} \sum_{i=1}^m \left(2(f_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot \frac{\partial}{\partial \mathbf{w}_j} (f_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)}) \right) \\&= \frac{1}{m} \sum_{i=1}^m \left((f_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot \frac{\partial}{\partial \mathbf{w}_j} \left(\sum_{k=0}^n \mathbf{w}_k \mathbf{x}_k^{(i)} - y^{(i)} \right) \right) \\&= \frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot \mathbf{x}_j^{(i)}\end{aligned}$$



通过计算 $\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = 0$ $\frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot \mathbf{x}_j^{(i)} = 0$

来获得 $L(\mathbf{w})$ 时的参数 $\mathbf{w}$ $\sum_{k=0}^n \sum_{i=1}^m \mathbf{w}_k \mathbf{x}_k^{(i)} \mathbf{x}_j^{(i)} = \sum_{i=1}^m (\mathbf{x}_j^{(i)} y^{(i)})$

左侧是个二次型，可以用向量表示上式：

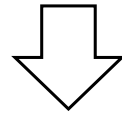
$$(X_{\cdot j}^T)_{(1 \times m)} X_{(m \times (n+1))} \mathbf{w}_{((n+1) \times 1)} = (X_{\cdot j}^T)_{(1 \times m)} \cdot \mathbf{y}_{(m \times 1)}$$

简略表示为： $X_{\cdot j}^T X \mathbf{w}^T = X_{\cdot j}^T \mathbf{y}$

$$X_{(m \times (n+1))} = \begin{bmatrix} (\mathbf{x}^{(1)})^T \\ (\mathbf{x}^{(2)})^T \\ \vdots \\ (\mathbf{x}^{(m)})^T \end{bmatrix}$$
$$X^T X \mathbf{w} = X^T \mathbf{y}$$

$\boldsymbol{\theta} = (X^T X)^{-1} X^T \mathbf{y}$ 其中， $X^T X$ 需要可逆 讨论算法软件实现的局限性

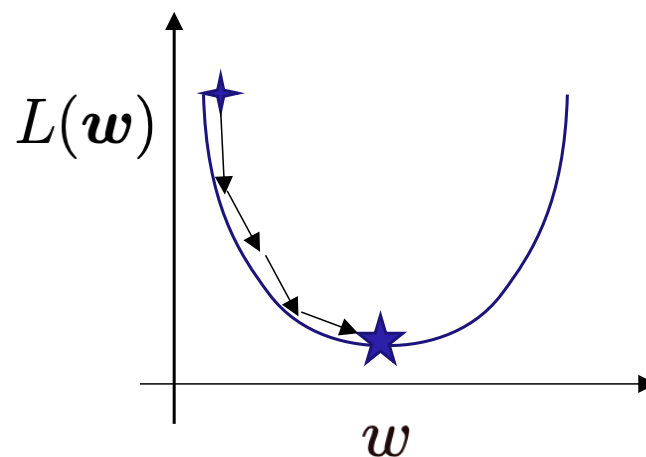
$$\sum_{k=0}^n \sum_{i=1}^m \mathbf{w}_k \mathbf{x}_k^{(i)} \mathbf{x}_j^{(i)} = \sum_{i=1}^m (\mathbf{x}_j^{(i)} y^{(i)})$$



$$\begin{aligned}
 j = 0 & : \quad \sum_{i=1}^m \mathbf{x}_0^{(i)} \mathbf{x}_0^{(i)} \mathbf{w}_0 + \sum_{i=1}^m \mathbf{x}_0^{(i)} \mathbf{x}_1^{(i)} \mathbf{w}_1 + \dots + \sum_{i=1}^m \mathbf{x}_0^{(i)} \mathbf{x}_n^{(i)} \mathbf{w}_n = \sum_{i=1}^m \mathbf{x}_0^{(i)} y^{(i)} \\
 j = 1 & : \quad \sum_{i=1}^m \mathbf{x}_1^{(i)} \mathbf{x}_0^{(i)} \mathbf{w}_0 + \sum_{i=1}^m \mathbf{x}_1^{(i)} \mathbf{x}_1^{(i)} \mathbf{w}_1 + \dots + \sum_{i=1}^m \mathbf{x}_1^{(i)} \mathbf{x}_n^{(i)} \mathbf{w}_n = \sum_{i=1}^m \mathbf{x}_1^{(i)} y^{(i)} \\
 & \dots \\
 j = n & : \quad \sum_{i=1}^m \mathbf{x}_n^{(i)} \mathbf{x}_0^{(i)} \mathbf{w}_0 + \sum_{i=1}^m \mathbf{x}_n^{(i)} \mathbf{x}_1^{(i)} \mathbf{w}_1 + \dots + \sum_{i=1}^m \mathbf{x}_n^{(i)} \mathbf{x}_n^{(i)} \mathbf{w}_n = \sum_{i=1}^m \mathbf{x}_n^{(i)} y^{(i)}
 \end{aligned} \tag{17}$$

4.2 优化 - 梯度下降法

- 【依据】：梯度的方向是函数增长最快的方向，负梯度的方向是代价函数下降最快的方向。（数学证明自行搜索）
- 计算一个函数的最小值，就可以从一个初始点，向着当前位置梯度方向逐渐下降的方法来做。



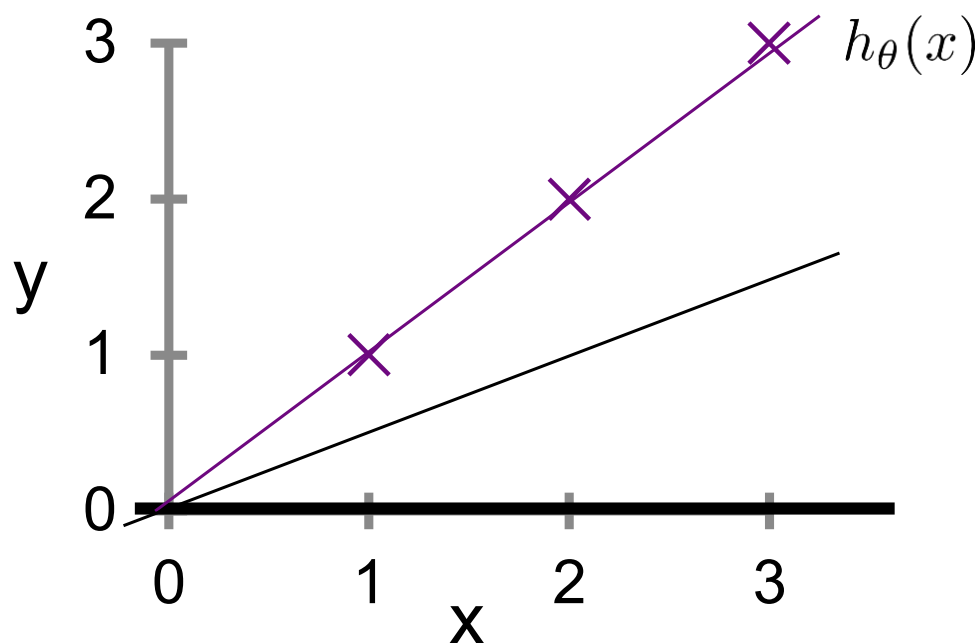
【注1】：损失函数是关于模型参数的函数

【注2】：模型参数根据负梯度进行小幅调整

4.2 优化-梯度下降法-演示1(样本空间与代价函数空间)

Predicting function: $f_w(x)$

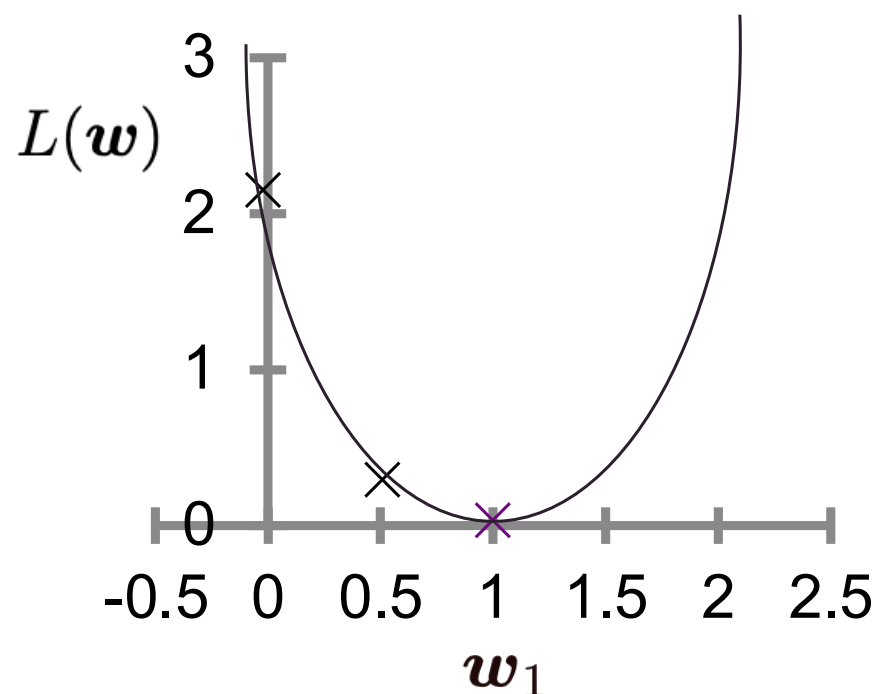
(设 $w_0 = 0$)



样本空间中三组参数对应的判决函数

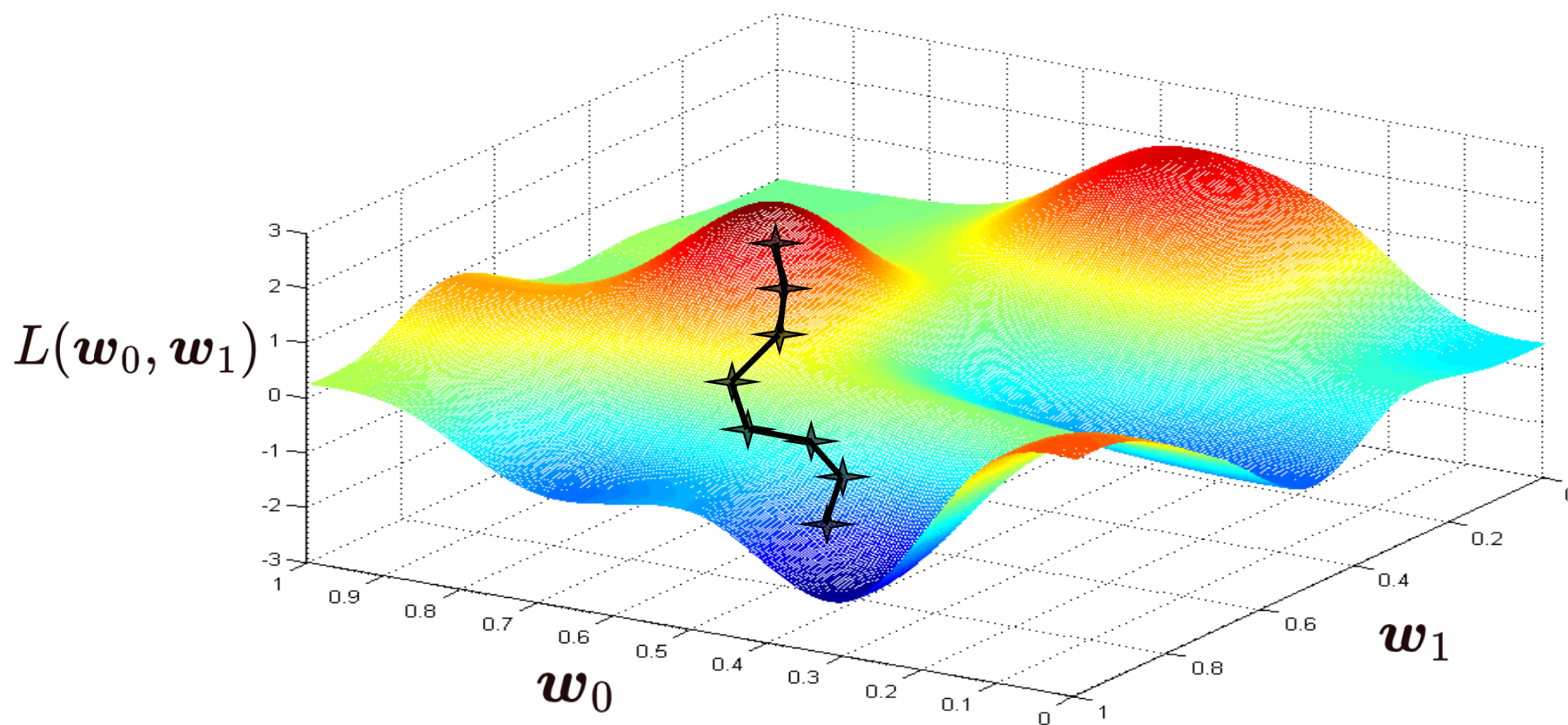
Loss function: $L(w)$

(function of the parameter w_1)



代价函数空间中三组参数对应的代价值

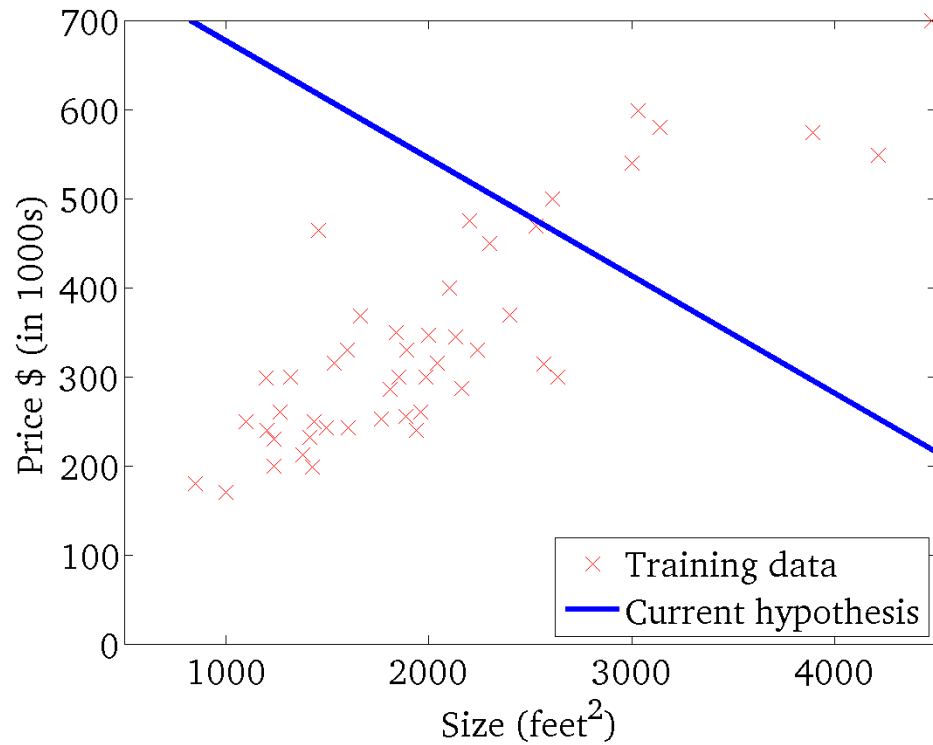
4.2 优化 - 梯度下降法-演示2 (二维参数)



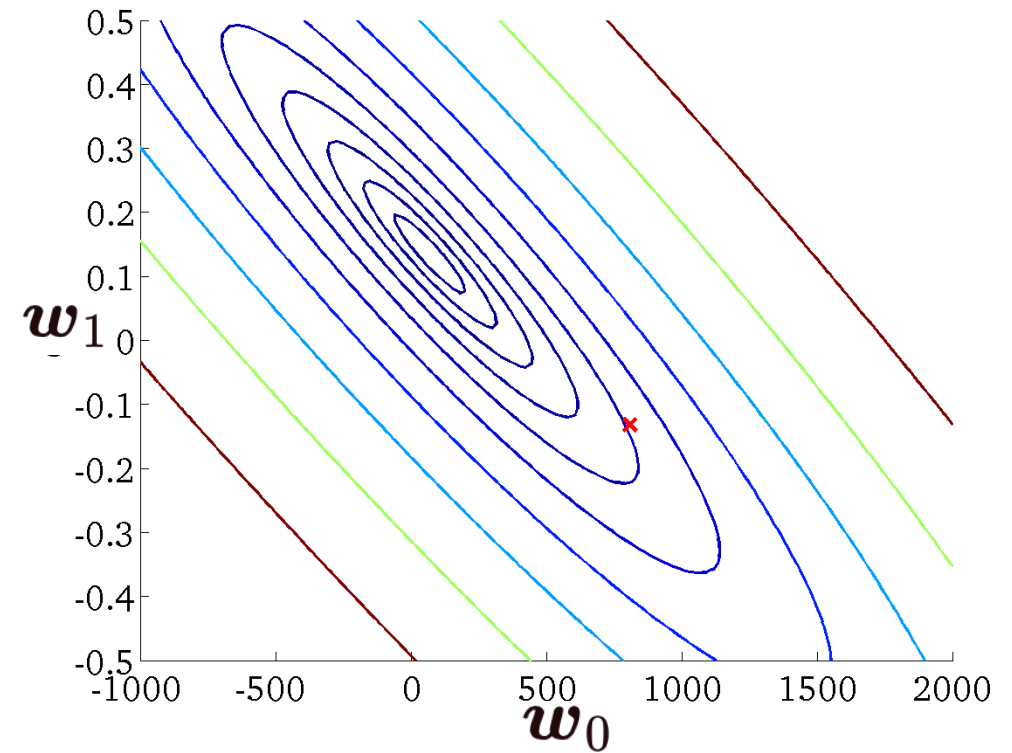
在梯度下降法中，每次迭代中根据梯度调整模型参数的示意

Contour Plot of LR's Cost Function

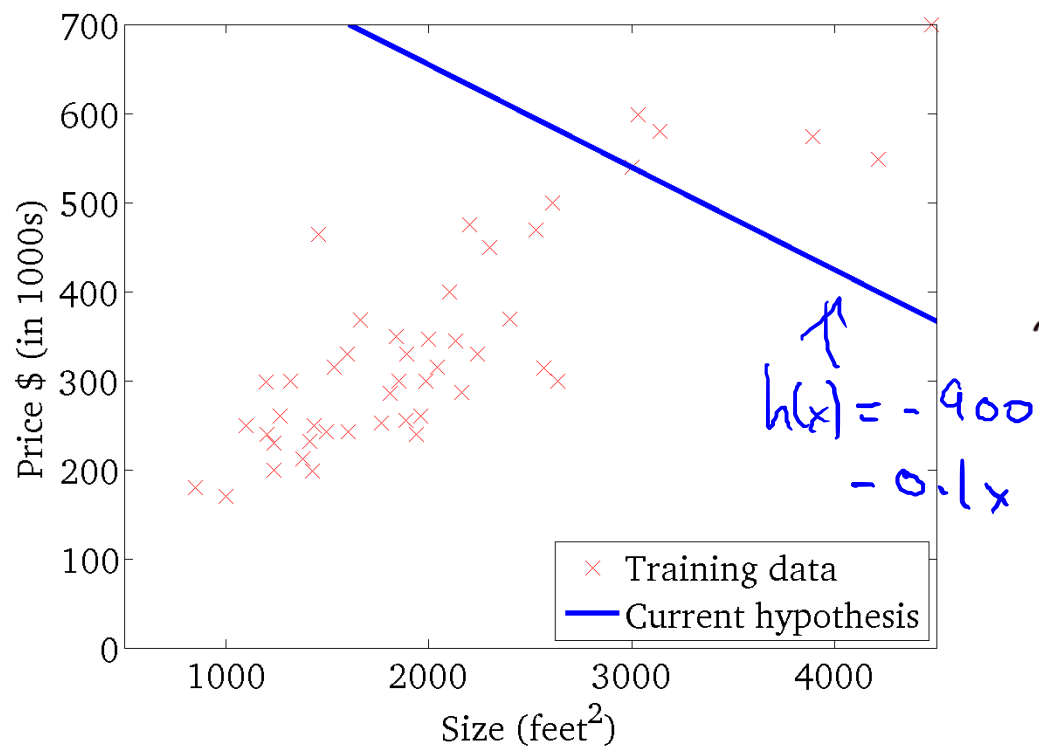
$$f_w(x)$$



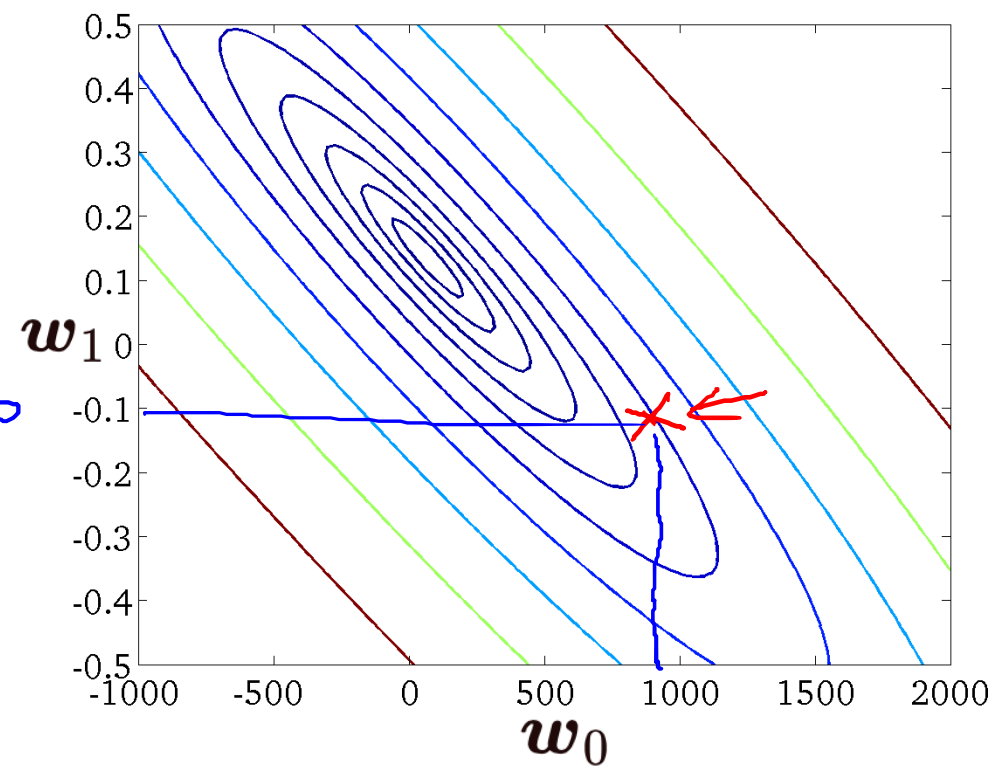
$$L(w)$$



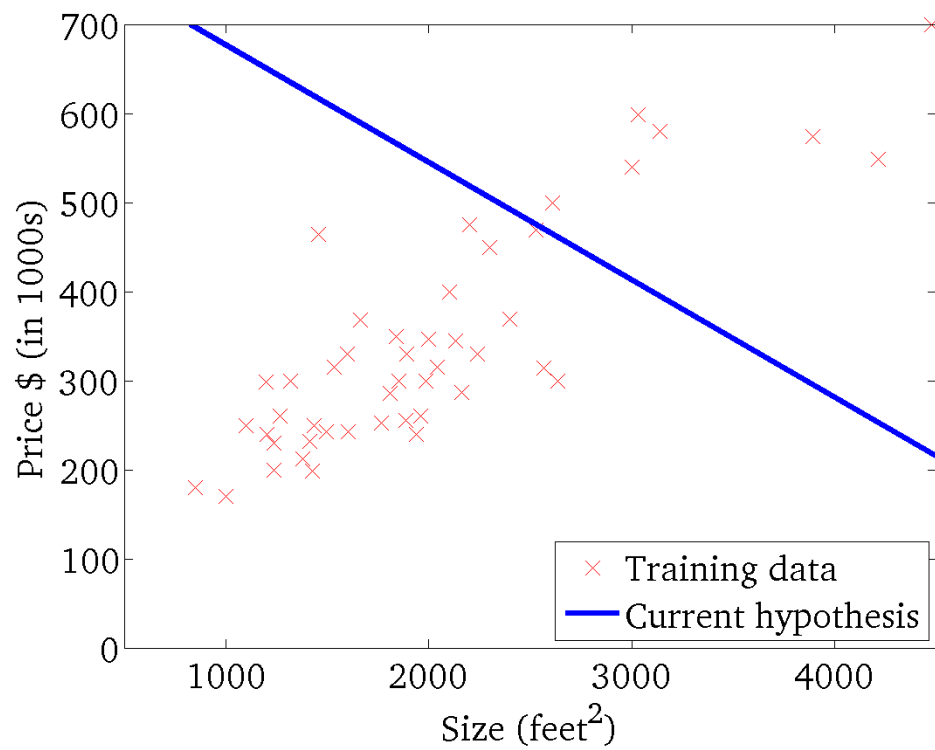
$$f_w(x)$$



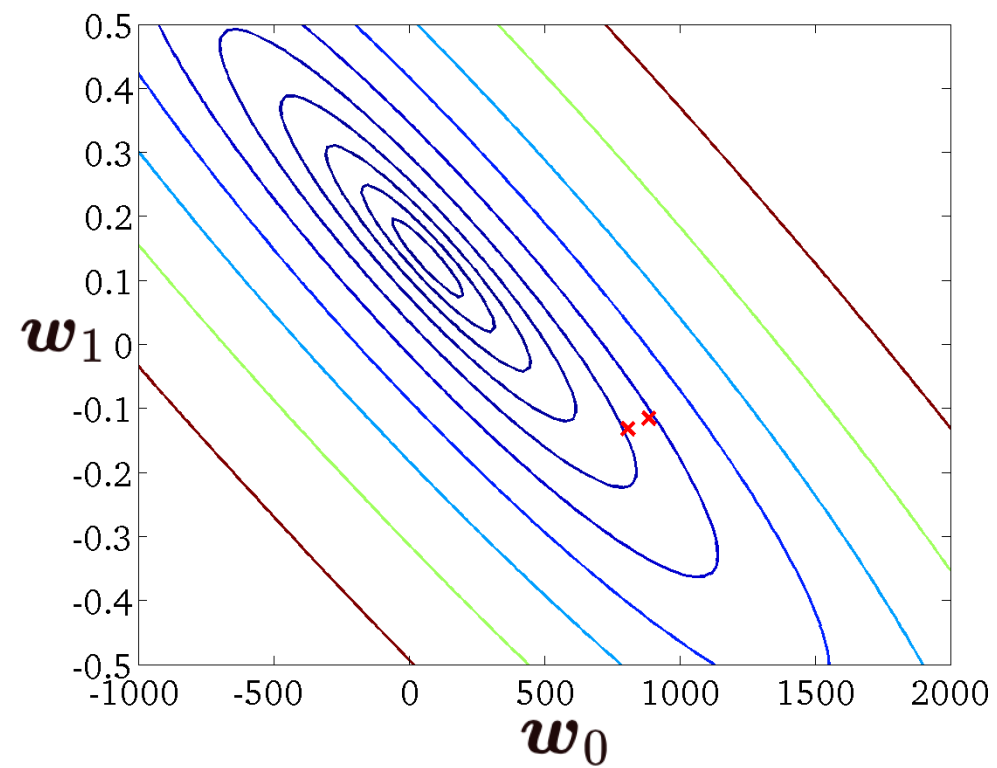
$$L(w)$$



$$f_w(\mathbf{x})$$



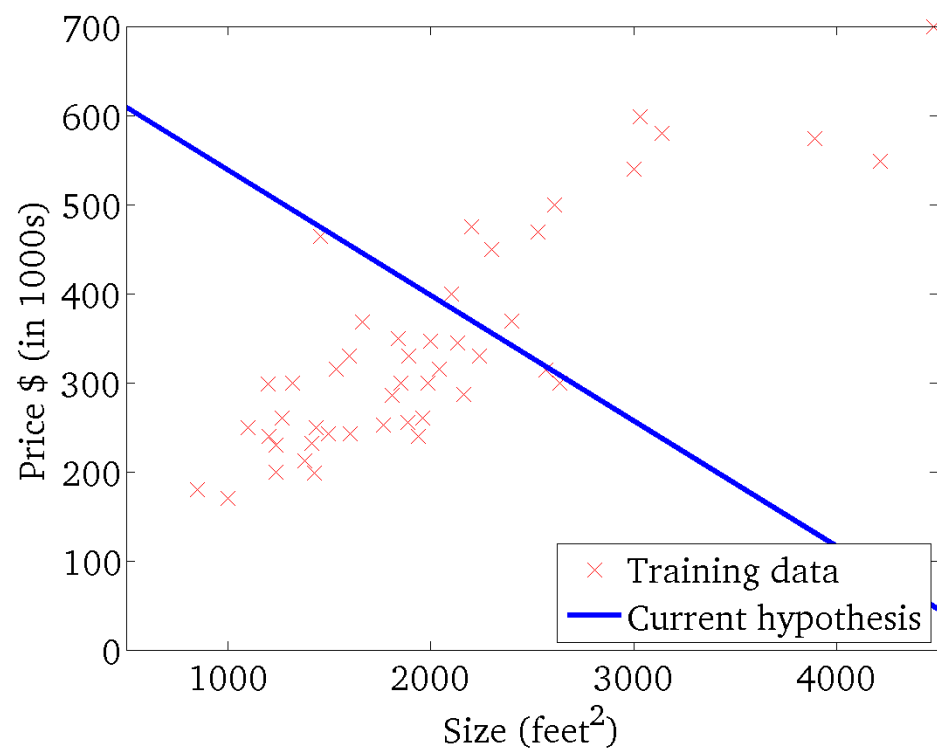
$$L(\mathbf{w})$$





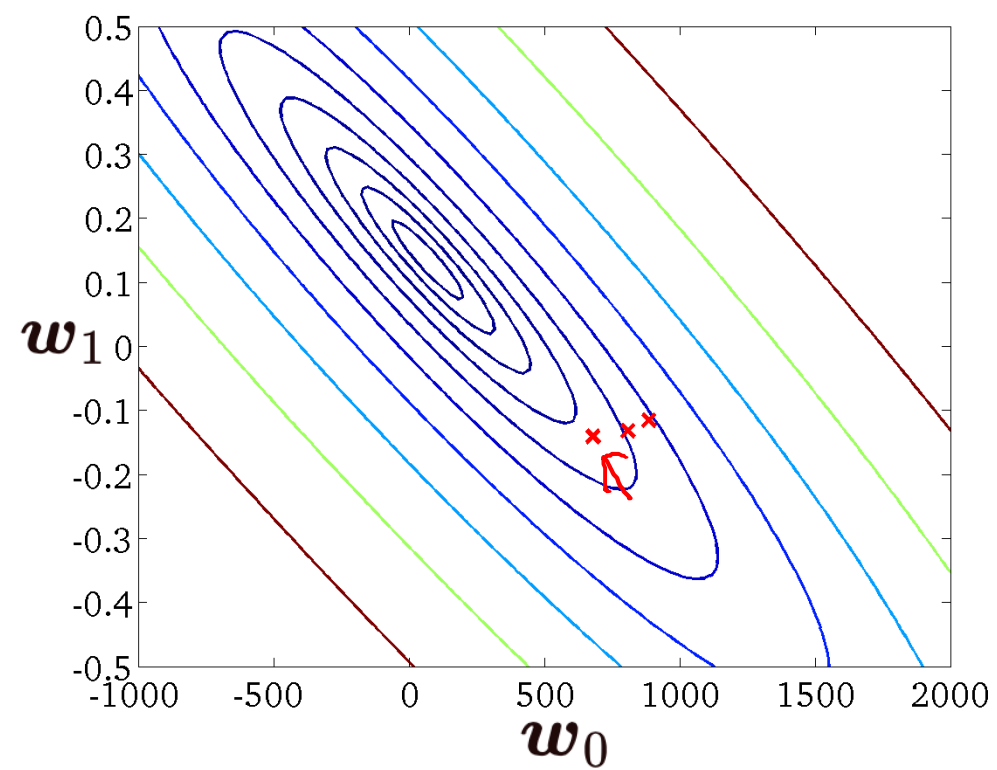
$$h_{\theta}(x)$$

$$\theta_0, \theta_1$$

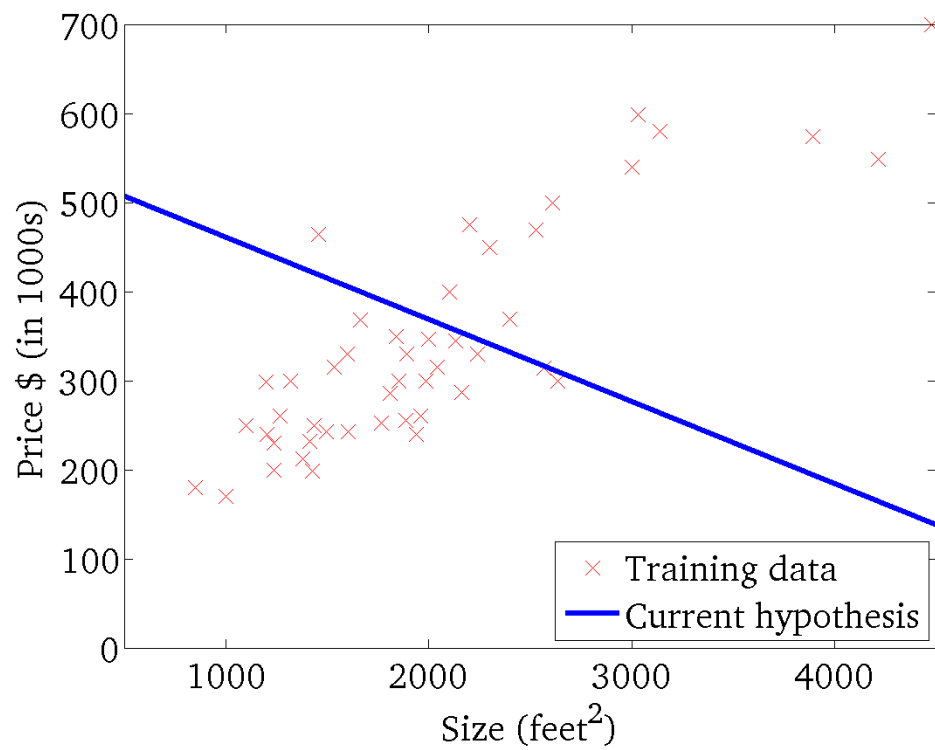


$$L(w)$$

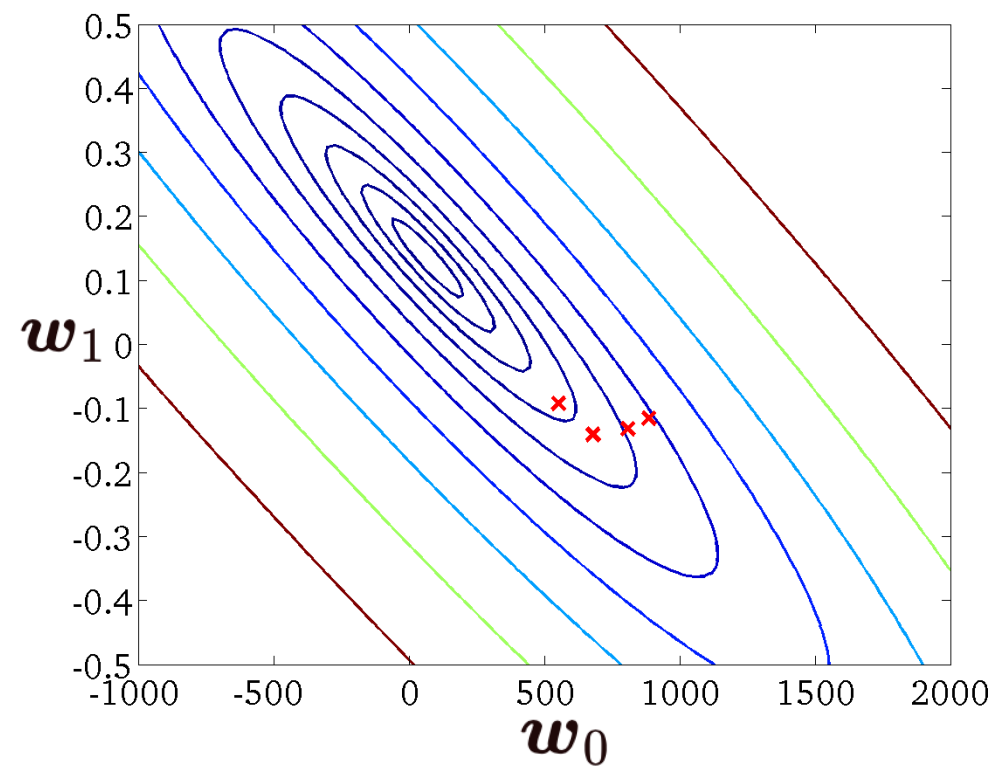
$$\theta_0, \theta_1$$



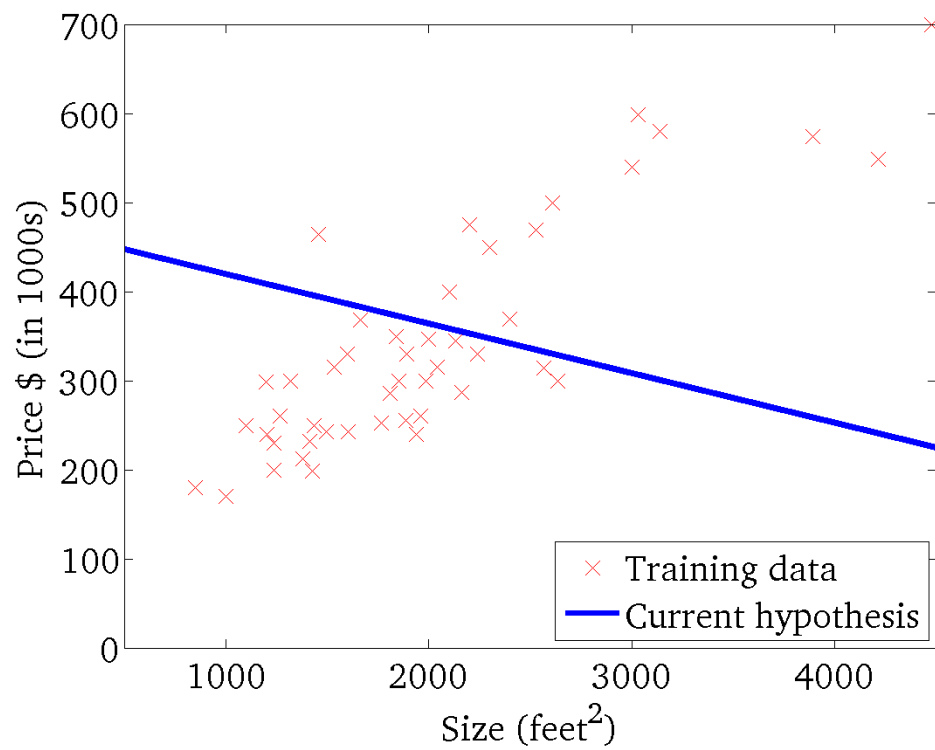
$$h_{\theta}(x)$$



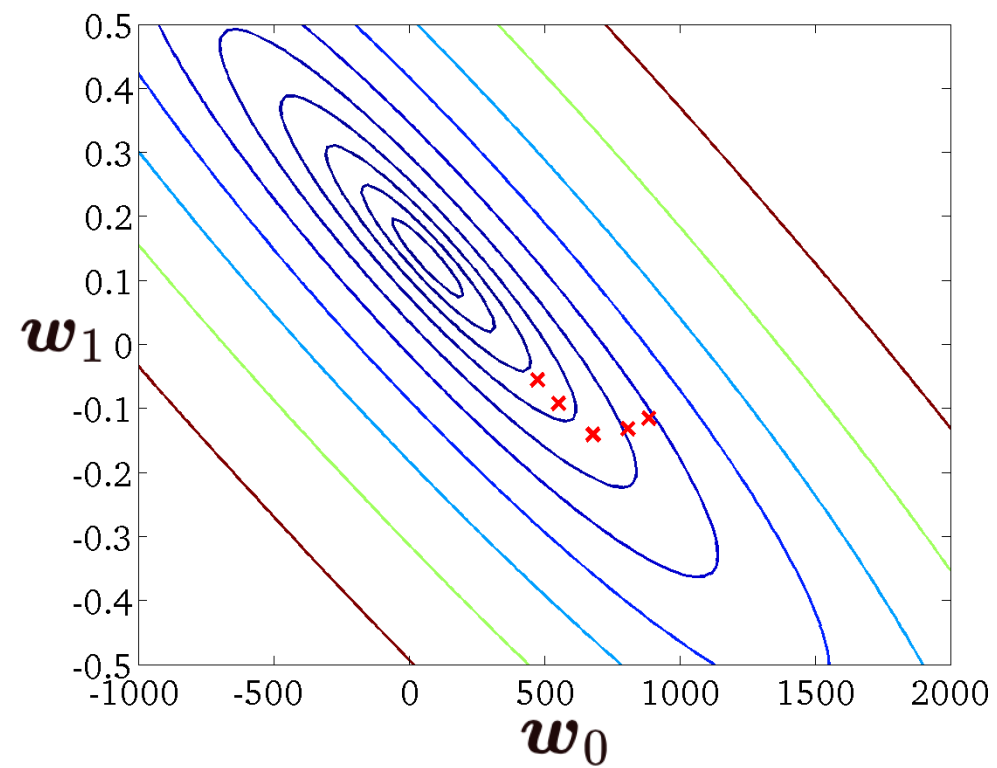
$$J(\theta_0, \theta_1)$$



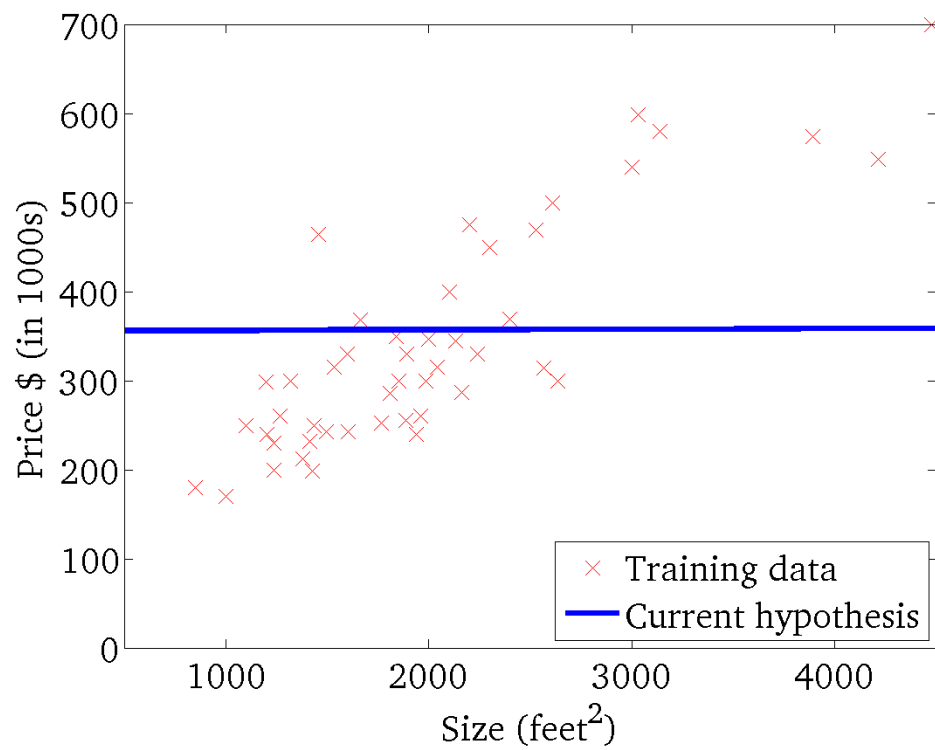
$$f_w(\mathbf{x})$$



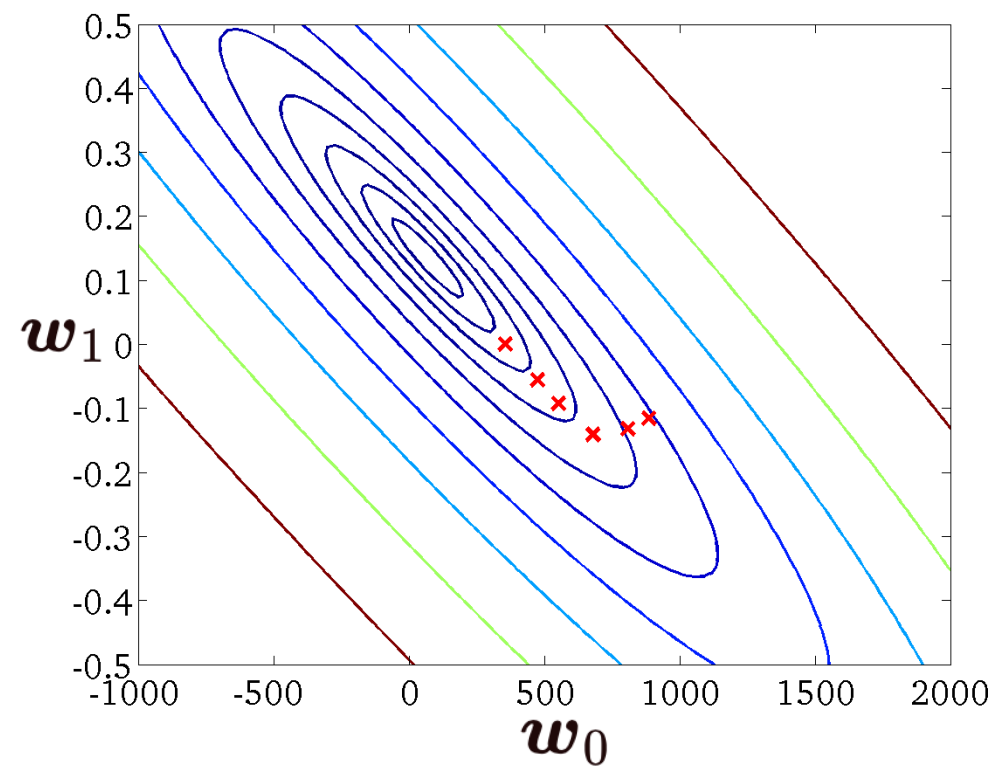
$$L(\mathbf{w})$$



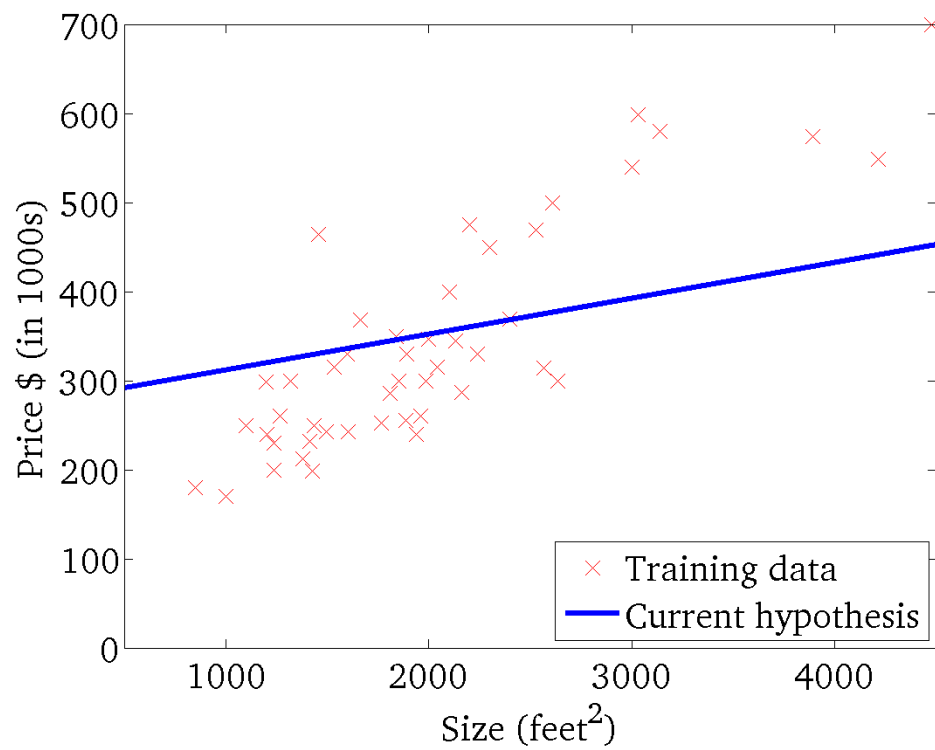
$$f_w(\mathbf{x})$$



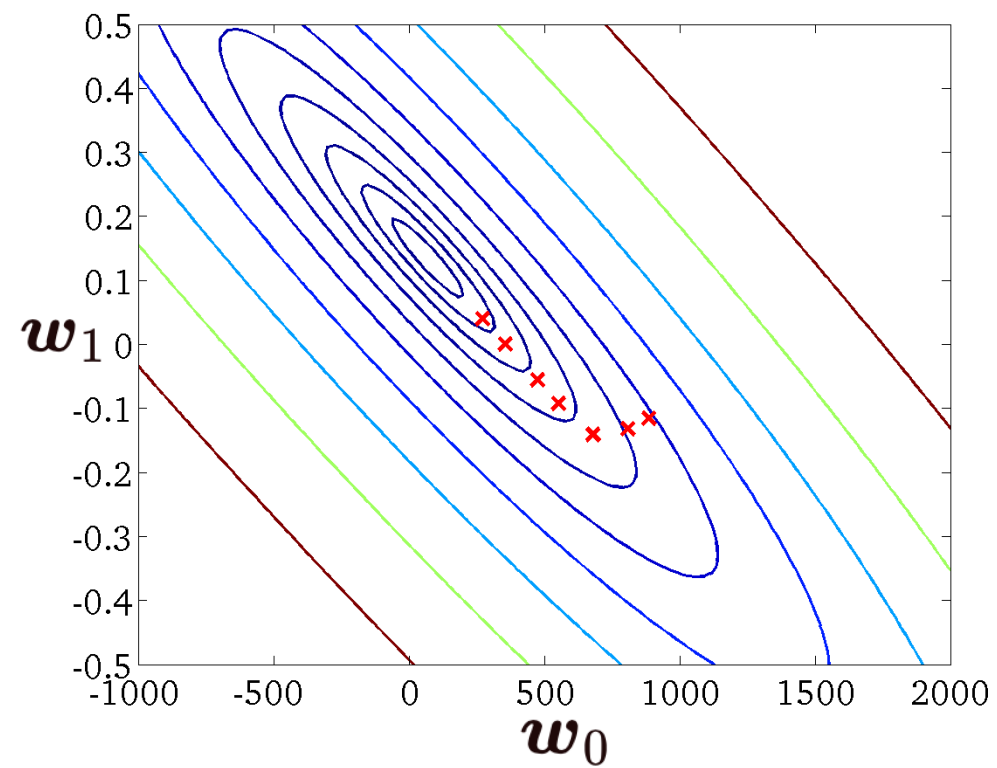
$$L(\mathbf{w})$$



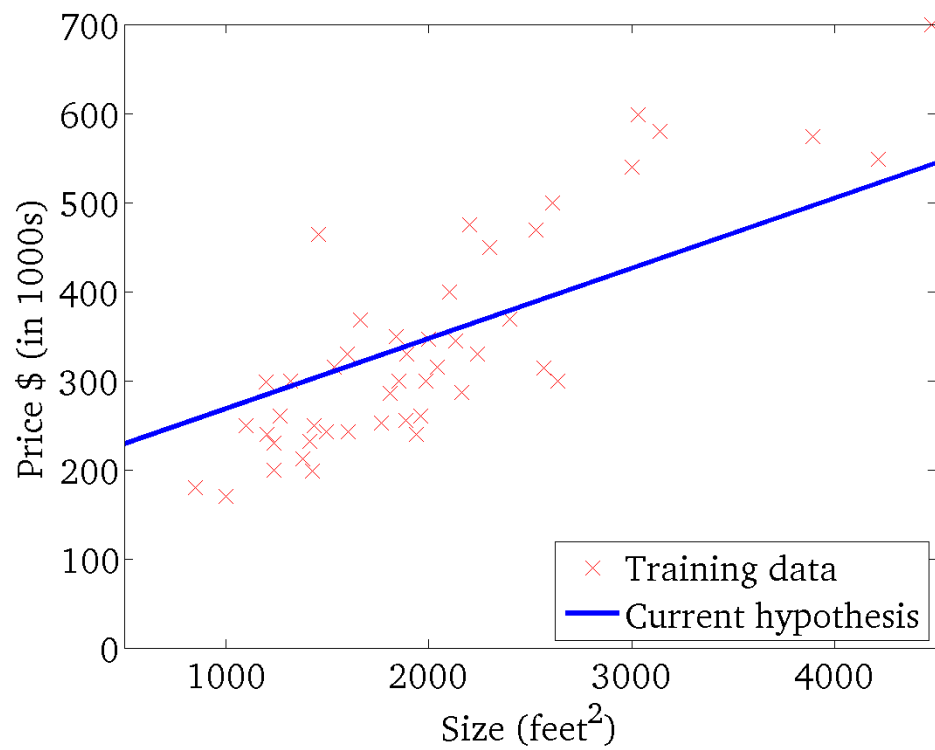
$$f_w(\mathbf{x})$$



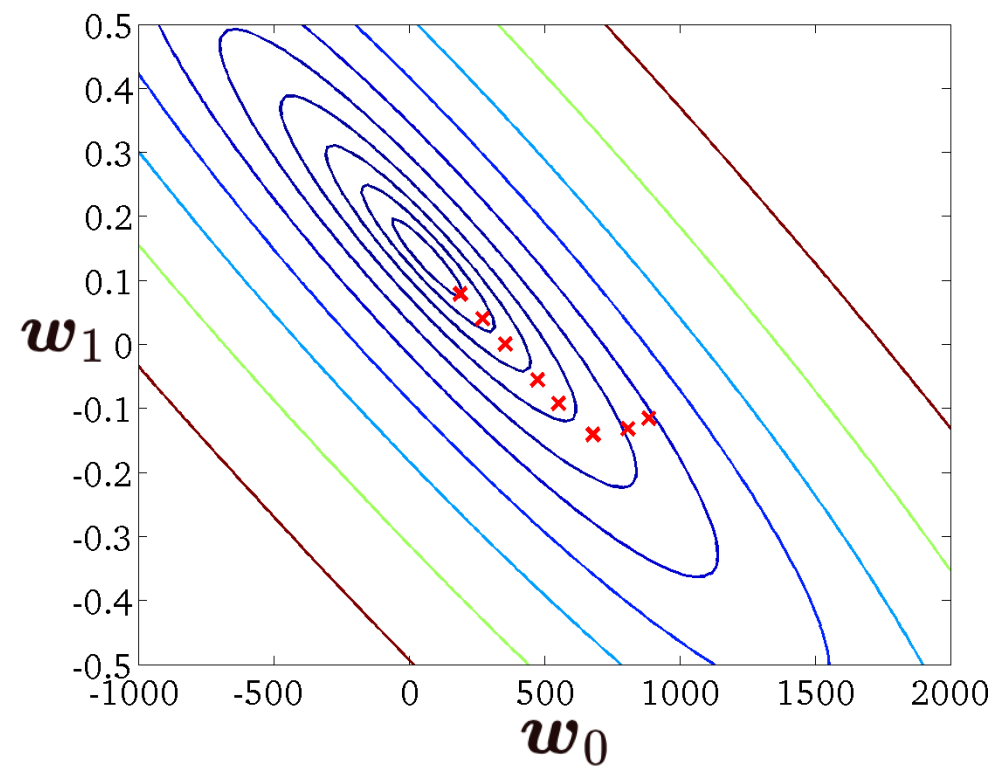
$$J(\theta_0, \theta_1)$$



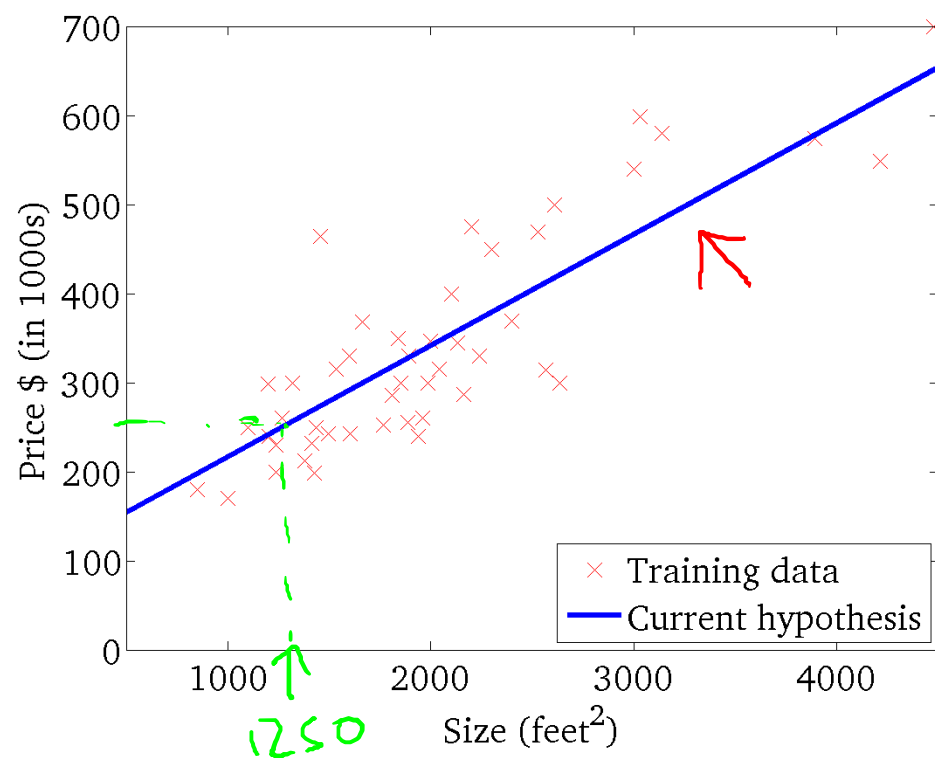
$$f_w(\mathbf{x})$$



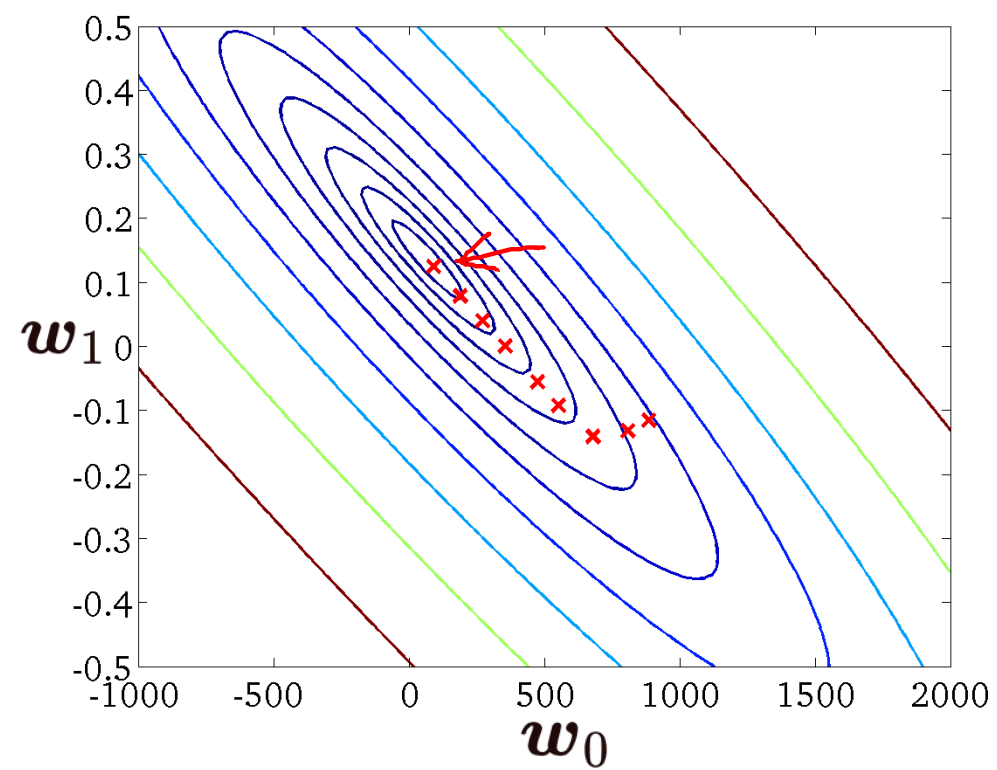
$$L(\mathbf{w})$$



$$f_w(x)$$



$$L(w)$$



4.2 优化-梯度下降法-代码实现

Gradient Descent Algorithm

Algorithm 1: 用于线性回归的梯度下降法

Input: 样本数据集 $\mathbf{X}$, $\mathbf{y}$, 最大迭代次数 T

Output: $\mathbf{w}$

```
1 随机初始化  $\mathbf{w}$ 
2 for  $t = 1, \dots, T$  do
3   for  $j = 1, \dots, n$  do
4      $\mathbf{w}_j := \mathbf{w}_j - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)}) \mathbf{x}_j^{(i)}$ 
5   end
6   if 迭代前后  $\mathbf{w}$  变化小于  $\varepsilon$  then
7     break
8   end
9 end
```

要同时更新 $\mathbf{w}_0$ 和 $\mathbf{w}_1$

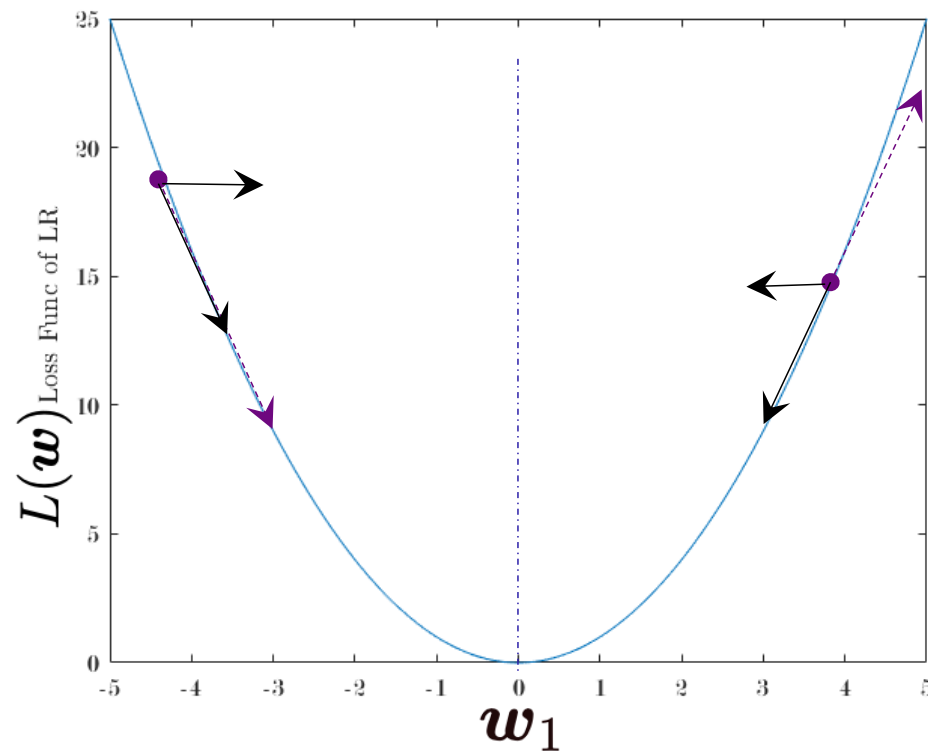
Batch: 每轮迭代都要使用所有训练样本

$$\text{Batch: } \frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)})$$

4.3 关于梯度下降算法的一些细节

- **梯度有正有负，对于负梯度一直向着最小值去有些不解**
- 怎么将梯度法下降扩展到多维特征上
- 初始值对算法的影响（&GD可以用到其他损失函数上）
- 步长对算法的影响

4.3.1 梯度有正有负，负梯度一直向着最小值去？



$$w_1 \leftarrow w_1 - \alpha \frac{\partial L(w)}{\partial w_1}$$

当 $\frac{\partial L(w_1)}{\partial w_1}$ 为正时, w_1 会减少

当 $\frac{\partial L(w_1)}{\partial w_1}$ 为负时, w_1 会增加

4.3 关于梯度下降算法的一些细节

- 梯度有正有负，对于负梯度一直向着最小值去有些不解
- **怎么将梯度法扩展到多个变量上**
- 初始值对算法的影响（&GD可以用到其他损失函数上）
- 步长对算法的影响

4.3.2 多元线性回归

前提假设：各个维度相互独立

多维线性回归模型：

$$f_w(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

损失函数：

$$L(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (f_w(\mathbf{x}^{(i)}) - y^{(i)})^2$$

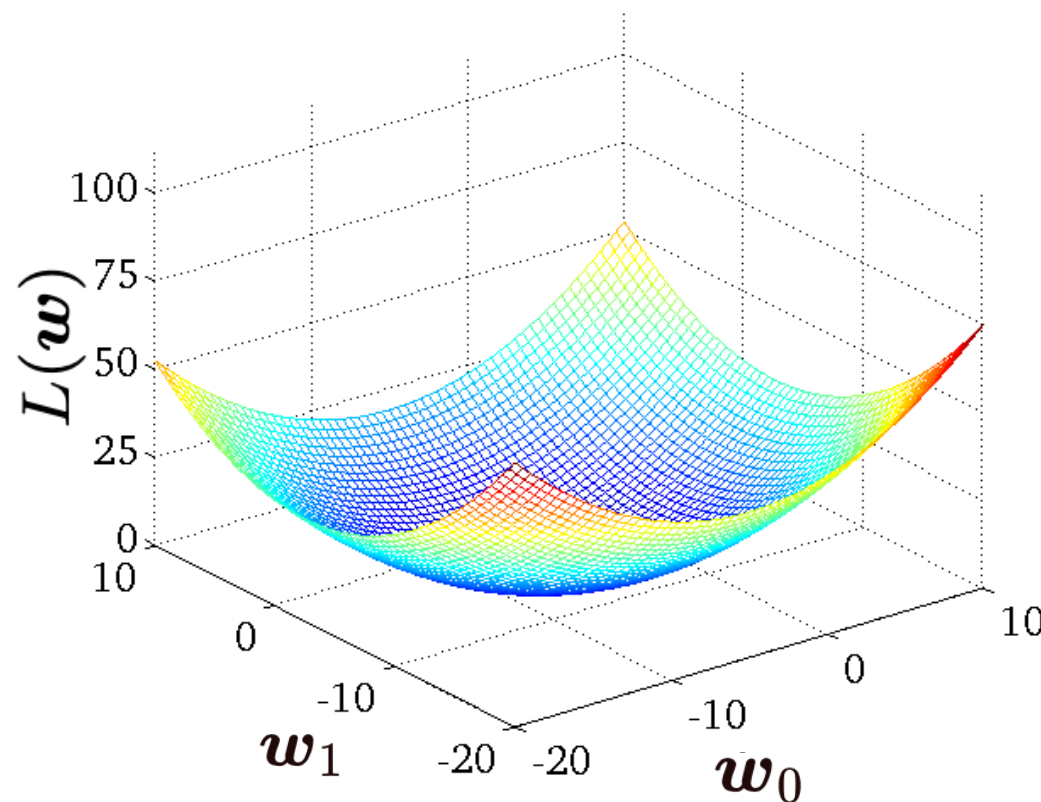
梯度下降法中，在原有算法基础上，从更新2维梯度变为更新 $n+1$ 维梯度。（因为各维度间相互独立，因此，各维梯度可以各自计算，相互之间没有影响。）

4.3 关于梯度下降算法的一些细节

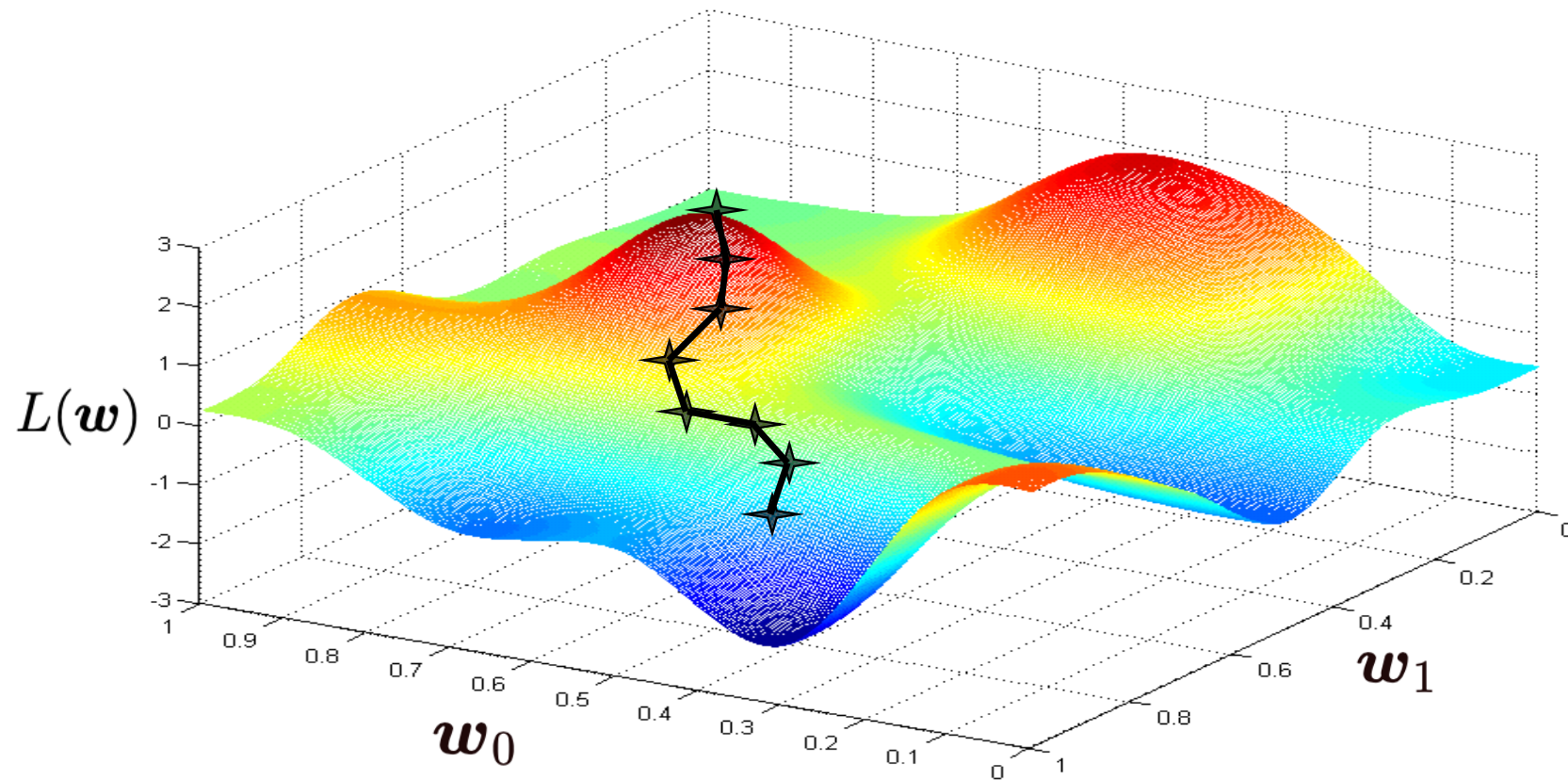
- 梯度有正有负，对于负梯度一直向着最小值去有些不解
- 怎么将梯度法下降扩展到多维特征上
- **初始值对算法的影响（GD可以用到其他损失函数上）**
- 步长对算法的影响

4.3.3 初始值对GD for LinearReg算法的影响

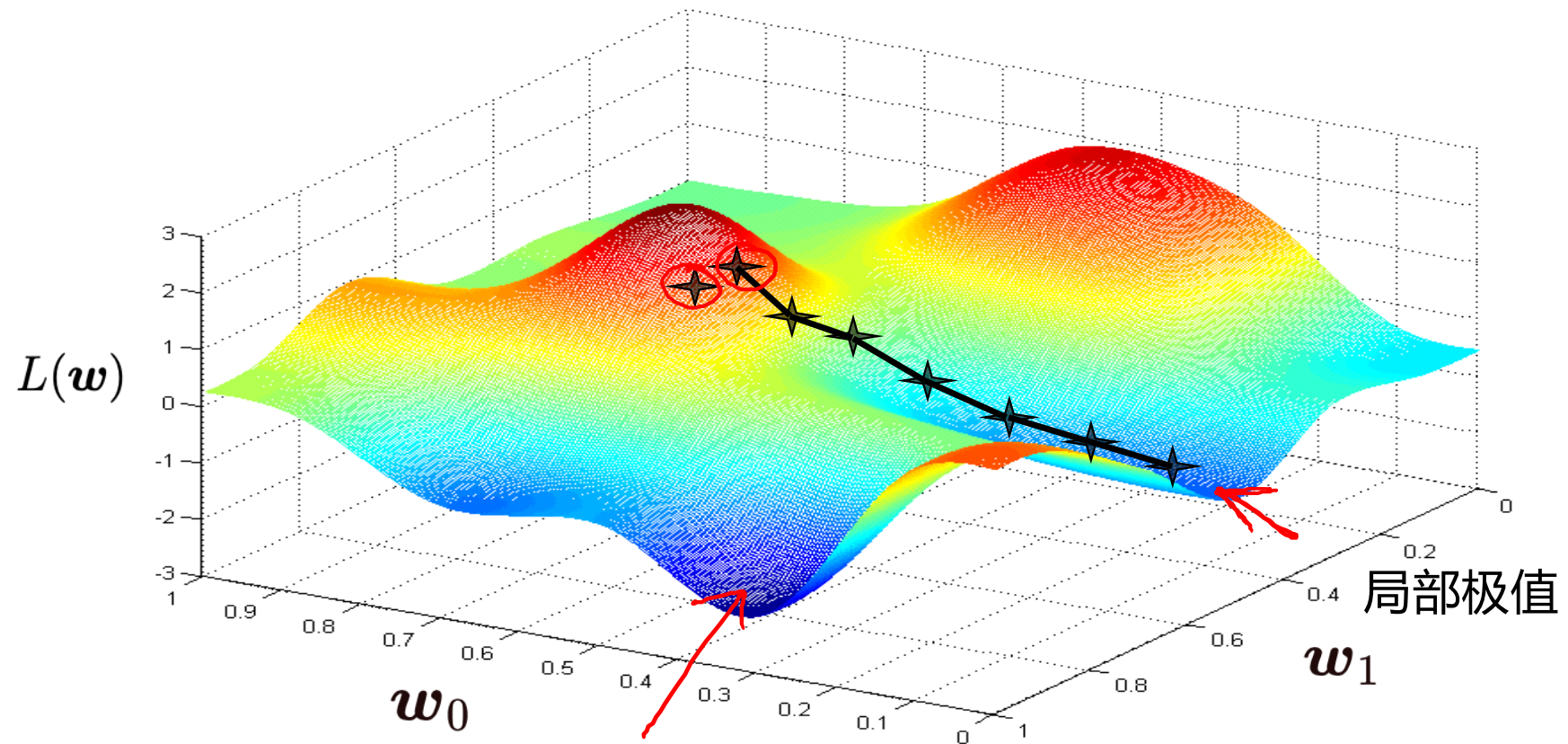
线性回归的损失函数正好是个碗型函数（凸函数，Convex Function）
 结论：初始值对结果影响不大，只是得到最优解的速度快慢而已



4.3.3 GD对其他代价函数的初值影响 1/2



4.3.3 GD对其他代价函数的初值影响 2/2



4.3 关于梯度下降算法的一些细节

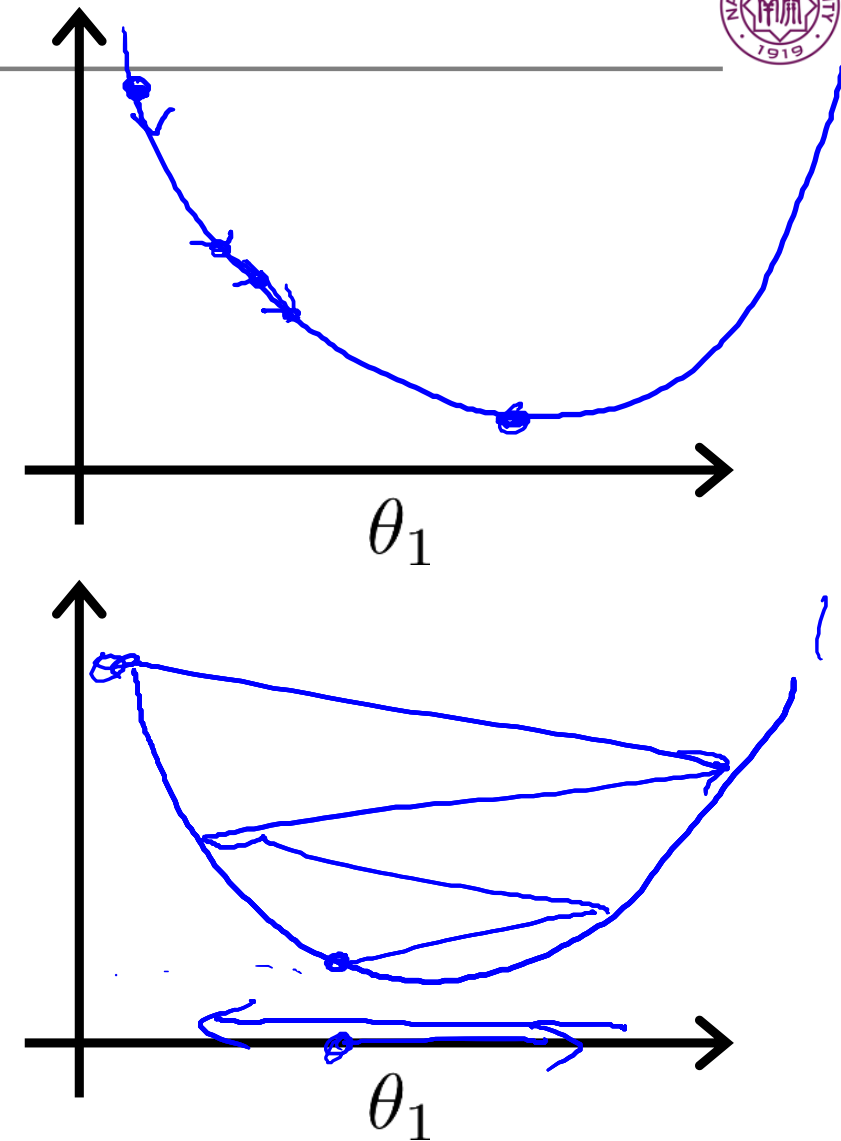
- 梯度有正有负，对于负梯度一直向着最小值去有些不解
- 怎么将梯度法下降扩展到多维特征上
- 初始值对算法的影响（&GD可以用到其他损失函数上）
- **步长对算法的影响**

4.3.4 步长或学习率对算法的影响

$$\boldsymbol{w} := \boldsymbol{w} - \alpha \frac{\partial L(\boldsymbol{w})}{\partial \boldsymbol{w}}$$

If α is too small, gradient descent can be slow.

If α is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.

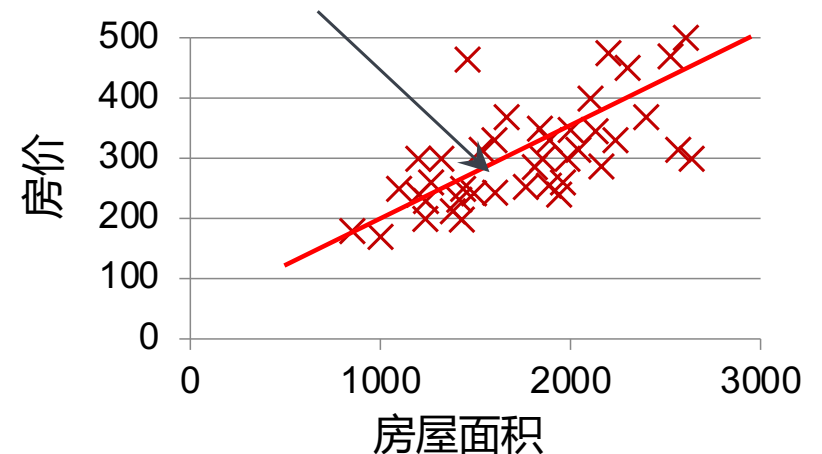


5. 从线性回归看机器学习模型

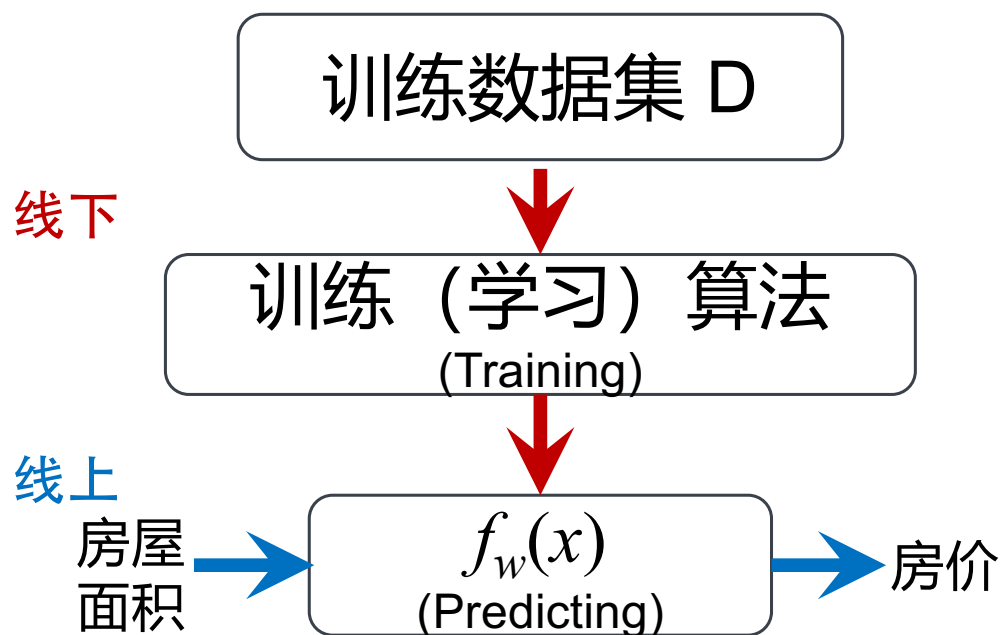
- **Machine Learning**[Tom Mitchell, 1997]: A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E
- 机器学习涉及
 - 标注数据（任务相关）
 - 学习算法（损失函数、提升）
 - 预测模型（Artificial Intelligence）

1. 任务、数据和模型
2. 模型的表示
3. 线性回归的代价函数
4. 优化：最小化代价函数

$$f_w(x) = w_0 + w_1x$$



5. 从线性回归看机器学习的训练和预测过程

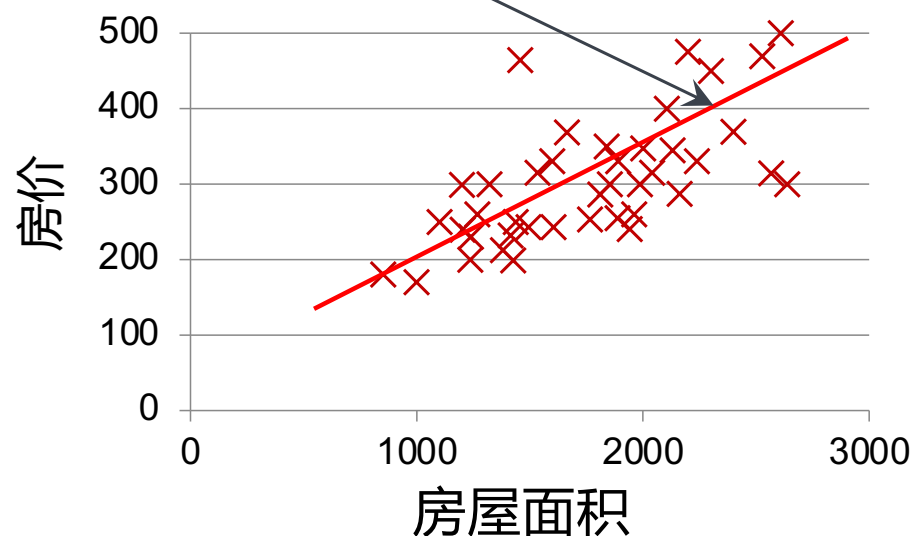


h (hypothesis, 假设, 预测模型): 是从房间大小到房价的一个关系映射

映射可能是线性关系, 也可能是指数、二次或其他非线性关系。

预测模型 h 的表示

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



需满足训练数据同测试数据的独立同分布假设

5. 从应用角度看机器学习的训练和预测过程（以华为昇腾为例）

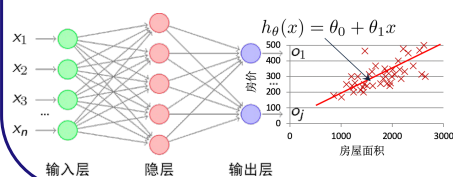
数据



线下训练用
服务器



线上实时预测



车自身状态
道路情况
周围物体情况

方向
给油量
刹车程度
速度
加速度

- 需要**提前**准备的训练过程（**线下**）

- 硬件：



- 软件：



CANN (CUDA)

- 实际应用中的预测（**线上**）

- 硬件：



5. 机器学习的范式

- 从训练数据中学习输入至输出间的映射规律（**训练**），使用模型及参数表征和记录学习到的规律；
- 利用学习到的规律对输入数据进行输出**预测**。其前提规律是待预测的数据分布同训练数据的分布是独立同分布；
- 统计机器学习是机器学习中的派别之一，其主要形式为使用**代价函数**度量模型现状与目标间的差距，并使用**优化**方法尽快缩小此差距。

- 任务
- 数据
- 线性模型
- 代价函数（损失函数）
- 优化方法
 - 解析求导法
 - 梯度下降法
- 优化方法（梯度下降）
 - 扩展至多元(维)线性回归
 - 学习步长
 - 适用范围及局部极小值
 - 初始值的影响
- 泛化至统计机器学习
 - 训练、预测（测试）
 - 代价函数与优化方法